

LAB 4

Training, Validating, and Deploying a Megalopa Detection Model on Raspberry Pi 5

Fork-based GitHub workflow | Google Colab | YOLO26n | Static image, recorded video, and USB webcam

Session	Duration	Focus	Marks
Session 1	2 hours	Fork, dataset inspection, training, evaluation, ONNX export, and Git push	60
Session 2	2 hours	Fork synchronization, CPU-only Pi setup, image/video/live validation, and deployment evaluation	40
Total	4 hours	Complete cloud-to-edge workflow	100

Official student submissions: SecXX_GroupXX_Lab4_Session1.pdf and SecXX_GroupXX_Lab4_Session2.pdf

1. Laboratory objective

Students complete a reproducible object-detection workflow from a labelled megalopa dataset to edge inference on Raspberry Pi 5. The instructor maintains a public upstream repository, and each student group works in its own public fork.

- Inspect and validate a YOLO bounding-box dataset and its class mapping.
- Create train, validation, and test subsets and train a lightweight YOLO26n model in Google Colab.
- Interpret precision, recall, mAP, confusion matrices, and confidence-related plots.
- Export the trained PyTorch model to ONNX and push selected outputs to the group fork.
- Synchronize the group fork with the updated instructor repository before Session 2.
- Create a CPU-only Raspberry Pi environment and verify that CUDA is not active.
- Evaluate pretrained and custom models on static images, recorded video, and a live USB webcam.
- Compare PyTorch and ONNX deployment behaviour under controlled settings.

2. GitHub architecture

The laboratory uses the following repository relationship:

WUStuLab upstream -> group fork -> Colab training/push -> fork synchronization -> Raspberry Pi 5 clone

2.1 Instructor upstream

- Repository: WUStuLab/lab4-megalopa.
- Contains official datasets, validation media, notebooks, baseline models, and Raspberry Pi scripts.
- Students do not require Write access to this repository.
- The upstream main branch remains instructor-controlled.

2.2 Group fork and synchronization rule

- Every student must have a GitHub account.
- One designated group member creates the public group fork and adds the remaining members as collaborators.
- Session 1 clones and pushes to the group fork.
- Student-generated files must be stored under student_work/.
- Before Session 2, the group selects Sync fork -> Update branch on GitHub.
- Session 2 clones the synchronized group fork on Raspberry Pi 5.
- Files under shared/ are instructor-managed and should not be edited by students.

Commit traceability: Record the Session 1 training commit and the newer synchronized Session 2 commit. After upstream synchronization, the fork HEAD may no longer equal the original Session 1 commit.

3. Material supplied through Google Classroom/GitHub

Material	Purpose
Lab4_Instruction.docx / .pdf	Complete two-session laboratory instruction
Session 1 Colab notebook	Dataset inspection, training, evaluation, export, answers, and PDF evidence
Session 2 Colab notebook	Pi workflow, static/video/live validation, evidence uploads, answers, and PDF evidence
YOLO box-label dataset	Shared model-training data
Validation images and videos	Controlled generalization and temporal validation
shared/models/yolo26n.pt	Instructor-supplied pretrained baseline model
shared/pi5/megalopa_detection_usb.py	Image, recorded-video, and USB-webcam inference on Raspberry Pi 5
requirements-pi files	Staged core and ONNX package guidance
Prepared Raspberry Pi 5 kit	Local screen, keyboard, mouse, internet, Git, Python venv support, and USB webcam

4. Session 1 - Dataset, training, evaluation, and fork push

Time	Activity
0-15 min	Student information, fork verification, and Colab setup
15-35 min	Dataset inspection and random five-image preview
35-50 min	Dataset split and training-parameter explanation
50-85 min	YOLO26n training
85-105 min	Metrics, plots, and model export
105-115 min	Image and video validation
115-120 min	Commit and push student_work/ to the group fork

4.1 Required Session 1 workflow

1. Enter group members, section, group number, fork owner, fork repository name, GitHub username, and commit email.
2. Install only the required Colab packages and verify the GPU runtime.
3. Clone the public group fork without authentication.
4. Inspect image counts, image sizes, image-label pairing, coordinate ranges, and class IDs.
5. Preview five random original/annotated image pairs; rerunning the cell must produce another sample.
6. Create the required train/validation/test split in temporary Colab storage.
7. Train YOLO26n using the instructor-defined parameters.
8. Display numerical metrics and all available YOLO plots.
9. Save best.pt, export ONNX, and verify both formats on the same image.
10. Validate the model on test images, supplied validation images, and the supplied validation video.
11. Stage only student_work/, authenticate through Colab Secrets, and push to the group fork.
12. Print the completed notebook as SecXX_GroupXX_Lab4_Session1.pdf.

4.2 GitHub authentication for the final push

Cloning a public fork is anonymous. Pushing requires authentication. The GitHub account used for the push must own the fork or be a collaborator.

- Create a fine-grained personal access token.
- Limit repository access to the group fork.
- Set Contents permission to Read and write.
- Store the token in Colab Secrets as GITHUB_TOKEN.
- Never paste the token into a visible notebook cell, output, screenshot, or PDF.

5. Session 2 - Raspberry Pi 5 deployment

The prepared kit contains Raspberry Pi OS, a local display, keyboard, mouse, internet access, Git, Python 3 with venv support, and a USB webcam. Students do not prepare an SD card and do not use SSH.

System package rule: Students must not run `sudo apt update`, `sudo apt upgrade`, or `sudo apt install` during the session. Those commands may trigger unrelated kernel/DKMS failures on a prepared Pi. System packages must be repaired and prepared by the instructor before class.

Time	Activity
0-10 min	Synchronize the group fork, clone it on the Pi, and verify supplied files
10-30 min	Create .venv; install CPU-only PyTorch/Torchvision and Ultralytics; verify environment
30-45 min	Scan webcam indexes and save a raw camera frame
45-60 min	Run pretrained YOLO26n baseline on the USB webcam
60-80 min	Validate the custom model on static images
80-100 min	Validate the custom model on recorded video and inspect saved outputs
100-110 min	Run live custom-model USB-webcam inference
110-117 min	Install ONNX Runtime; compare .pt and ONNX; test confidence and temperature
117-120 min	Complete evidence and export the Session 2 PDF

5.1 Required Session 2 workflow

1. Open the group fork on GitHub and select Sync fork -> Update branch.
2. Clone the synchronized public group fork into ~/lab4-megalopa-student.
3. Record the group fork URL, the Session 1 training commit, and the synchronized Session 2 commit.
4. Verify shared/models/yolo26n.pt, the updated deployment script, custom .pt/.onnx models, validation images, and at least one validation video.
5. Create a clean project .venv without using apt.
6. Install torch and torchvision from the official CPU-only PyTorch index before installing Ultralytics.
7. Verify CUDA available: False, PyTorch CUDA version: None, and CPU-only environment: PASSED.
8. Do not treat nvidia-ml-py as a CUDA runtime error; it is a small monitoring dependency and may remain installed.
9. Run --scan-cameras and record a camera index that returns a valid frame.
10. Run the pretrained baseline using shared/models/yolo26n.pt and save evidence.
11. Validate the custom model on a supplied static image before live testing.
12. Validate the custom model on a supplied recorded video and inspect the annotated video, JSON summary, and frame-metrics CSV.
13. Run the custom model with the live USB webcam and save at least one annotated frame.
14. Install ONNX Runtime only when the .pt versus ONNX comparison begins.
15. Repeat one controlled test with another confidence threshold and record the final Raspberry Pi temperature.
16. Upload evidence to the notebook and print it as SecXX_GroupXX_Lab4_Session2.pdf.

5.2 Static-image display behaviour

- The preview uses a movable OpenCV WINDOW_NORMAL window.
- The preview is positioned near the top-left corner and fitted to approximately 760 x 420 pixels.
- Saved annotated images retain their full output resolution.
- Press Q, Esc, Enter, or Space to close a static preview.
- Use --no-display when only saved outputs are required or the desktop window is not usable.

5.3 Recorded-video validation outputs

For each validation video, the deployment script produces the following evidence under student_work/results/pi5/custom/:

Output	Evidence provided
video_<name>_annotated.mp4 or .avi	Frame-by-frame model predictions over the recorded clip
video_<name>_summary.json	Source FPS, frames processed, average latency, inference FPS, end-to-end FPS, and count statistics
video_<name>_frame_metrics.csv	Per-frame timestamp, total detections, class counts, latency, and rolling inference FPS

5.4 Key Session 2 commands

The notebook contains the complete commands. These shortened forms identify the required operating modes:

Purpose	Command pattern
Scan cameras	<code>python shared/pi5/megalopa_detection_usb.py --scan-cameras</code>
Baseline webcam	<code>python shared/pi5/megalopa_detection_usb.py --baseline --mode camera --camera-index N</code>
Static image	<code>python shared/pi5/megalopa_detection_usb.py --mode image --image-index 1</code>
Recorded video	<code>python shared/pi5/megalopa_detection_usb.py --mode video --video-index 1</code>
Custom webcam	<code>python shared/pi5/megalopa_detection_usb.py --mode camera --camera-index N</code>
Non-graphical run	Add <code>--no-display</code> to image or video mode

6. Assessment

Deliverable	Questions	Marks
Session 1 completed Colab PDF	27 embedded questions	60
Session 2 completed Colab PDF	20 embedded questions	40
Total	47 embedded questions	100

6.1 Evidence policy

Questions are marked together with the relevant output. An answer unsupported by the required code output, metric plot, screenshot, photograph, saved image, annotated video, JSON summary, CSV record, or result table may receive reduced credit.

7. Instructor preparation checklist

- Insert the final dataset, validation images, and at least one short validation video.
- Verify every image-label pair and the intended class mapping.
- Test the full Session 1 notebook in a temporary student fork.
- Repair any pending apt/dpkg/DKMS errors before class; students must not repair system packages during the lab.
- Confirm `python3 -m venv` works and the CPU-only PyTorch index is reachable.
- Verify that the CPU-only check passes even when `nvidia-ml-py` is installed.
- Confirm `shared/models/yolo26n.pt` and the updated deployment script are committed to upstream main.
- Test `--scan-cameras` on every Pi kit and record at least one working USB-webcam index.
- Test the 760 x 420 preview, window movement, keyboard controls, and `--no-display` mode.
- Test static-image, recorded-video, live-webcam, .pt, and ONNX inference on the actual Pi 5 kits.
- Confirm that video validation produces an annotated video, JSON summary, and frame-metrics CSV.
- Freeze the upstream main branch after the final update and tell each group to synchronize its fork.
- Keep backup trained models, one backup USB webcam, and one backup Pi kit available.

8. Submission checklist

- Session 1 PDF contains all 27 answers, visible outputs, the group fork URL, and the Session 1 training commit ID.
- Session 2 PDF contains all 20 answers, the synchronized commit ID, CPU-only verification, and the working camera index.
- Session 2 evidence includes the baseline, static custom-model result, recorded-video result, live-webcam result, and .pt/ONNX comparison.
- The annotated video, JSON summary, and frame-metrics CSV paths are recorded in the Session 2 notebook.
- No token, password, or private credential appears in either PDF.
- The custom model files are visible in the group fork under student_work/models/.
- Student outputs remain under student_work/; instructor-managed shared/ files are unchanged in the group fork.

9. Common Session 2 error interpretation

Observation	Correct interpretation/action
nvidia-ml-py is listed	Allowed. Confirm torch.cuda.is_available() is False and torch.version.cuda is None.
8812au/DKMS or dpkg error during apt	System-level driver problem. Stop student installation and ask the instructor to repair the Pi.
No /dev/video0	Run --scan-cameras; the working USB webcam may use another index.
Static image window appears frozen/off-screen	Use the v9 script with fitted WINDOW_NORMAL preview, or run with --no-display.
No validation video is listed	The instructor repository is incomplete or the fork was not synchronized.
--video-index or --scan-cameras is unrecognized	The group fork contains an older deployment script; synchronize the fork and clone/pull again.

10. Reference links

- **GitHub - Fork a repository:** <https://docs.github.com/en/pull-requests/how-tos/work-with-forks/fork-a-repo>
- **GitHub - Syncing a fork:** <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/syncing-a-fork>
- **GitHub - Personal access tokens:** <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>
- **PyTorch - Start locally:** <https://pytorch.org/get-started/locally/>
- **Ultralytics - Quickstart:** <https://docs.ultralytics.com/quickstart/>
- **Ultralytics - Model export:** <https://docs.ultralytics.com/modes/export/>
- **Ultralytics - Raspberry Pi guide:** <https://docs.ultralytics.com/guides/raspberry-pi/>